

1. INTRODUCCIÓN

El presente proyecto tiene como objetivo construir un pipeline completo de clasificación de texto en español, integrando técnicas de Procesamiento de Lenguaje Natural y Deep Learning. El flujo de trabajo incluye selección del dataset, análisis exploratorio, limpieza textual, tokenización, comparación de procesamiento con NLTK y spaCy, representación numérica mediante TF-IDF y entrenamiento de una red neuronal en PyTorch.

2. DATASET

Para este proyecto se seleccionó un dataset de clasificación de sentimiento en español disponible en Hugging Face. La elección se justifica porque cumple con los requisitos de la consigna: contiene textos en español, posee etiquetas de clasificación y cuenta con más de 2.000 registros, lo que permite aplicar técnicas de aprendizaje automático y deep learning.

3. ANÁLISIS EXPLORATORIO DE DATOS

Durante el EDA se analizó la cantidad total de registros, la distribución de clases y la longitud de los textos. También se calcularon las palabras más frecuentes luego de aplicar una primera limpieza textual y eliminar stopwords. Este análisis permitió conocer el balance del dataset y detectar patrones generales en el vocabulario.

4. PREPROCESAMIENTO

El preprocesamiento incluyó limpieza con expresiones regulares para eliminar URLs, menciones, hashtags, caracteres especiales y espacios duplicados. Luego se aplicó tokenización y normalización textual con dos enfoques:

- NLTK: se utilizó tokenización y SnowballStemmer para español como aproximación de normalización léxica, ya que NLTK no posee un lematizador robusto integrado para español.
- spaCy: se utilizó el modelo `es_core_news_sm`, que permite tokenización y lematización real en español.

La comparación mostró que spaCy conserva mejor el significado lingüístico de las palabras, mientras que NLTK reduce las palabras a raíces que pueden ser menos interpretables.

5. REPRESENTACIÓN NUMÉRICA CON TF-IDF

Se utilizó `TfidfVectorizer` para transformar los textos procesados en vectores numéricos. TF-IDF resulta más adecuado que el simple conteo de palabras porque no solo considera la frecuencia de una palabra dentro de un documento, sino también qué tan importante es esa palabra dentro del conjunto total de documentos. De esta forma, palabras demasiado comunes reciben menor peso y términos más representativos reciben mayor importancia.

6. MODELO DE DEEP LEARNING

Se implementó una clase `TextClassifier` en PyTorch, heredando de `nn.Module`. El modelo recibe como entrada los vectores TF-IDF y utiliza capas densas fully connected. La arquitectura incluye:

- Primera capa densa.
- Batch Normalization.
- Función de activación ReLU.
- Dropout con $p = 0.5$.
- Segunda capa densa.
- Batch Normalization.
- Dropout con $p = 0.5$.
- Capa final de salida para clasificación.

7. ENTRENAMIENTO

El modelo fue entrenado utilizando el optimizador Adam y la función de pérdida `CrossEntropyLoss`. Además, se implementó Early Stopping con paciencia de 5 épocas, deteniendo el entrenamiento cuando la pérdida de validación no mejora durante 5 épocas consecutivas. Esto ayuda a evitar sobreajuste y conservar el mejor modelo encontrado durante el entrenamiento.

8. RESULTADOS

Los resultados se evaluaron sobre el conjunto de test utilizando accuracy, precision, recall, f1-score y matriz de confusión. Estas métricas permiten analizar no solo el rendimiento general del modelo, sino también su comportamiento por clase.

9. RESPUESTAS DE REFLEXIÓN

¿Por qué TF-IDF resulta más útil que el simple conteo de palabras?

TF-IDF resulta más útil porque asigna mayor importancia a las palabras que son frecuentes en un documento pero no tan comunes en todo el corpus. El conteo simple puede dar demasiado peso a palabras repetidas pero poco informativas. En cambio, TF-IDF ayuda a destacar términos más representativos para la clasificación.

¿Qué diferencias observaste entre la lematización con NLTK y spaCy?

NLTK fue utilizado con `SnowballStemmer` como aproximación de normalización para español. Este método reduce las palabras a raíces, pero puede generar términos menos interpretables. spaCy, en cambio, realiza lematización real, devolviendo formas base más cercanas al lenguaje natural. Por eso spaCy suele ser más adecuado cuando se busca conservar significado lingüístico.

¿Qué limitaciones encontraste en el desempeño del modelo?

Una limitación es que el modelo trabaja sobre vectores TF-IDF, que no capturan completamente el contexto ni el orden profundo de las palabras. Además, si existen clases

desbalanceadas, el modelo puede aprender mejor las clases mayoritarias. Otra limitación es que una red densa simple no captura relaciones semánticas complejas como lo haría un modelo basado en transformers.

¿Qué mejoras implementarías en una versión futura?

En una versión futura se podrían probar embeddings preentrenados en español, modelos transformer como BETO o RoBERTuito, ajuste de hiperparámetros, balanceo de clases, validación cruzada y una limpieza textual más específica según el dominio del dataset. También se podría comparar el rendimiento con modelos clásicos como Logistic Regression, SVM o Random Forest.

10. CONCLUSIÓN

El proyecto permitió construir un flujo completo de clasificación de texto en español, desde la limpieza y exploración del dataset hasta la representación numérica y el entrenamiento de una red neuronal. Se logró aplicar correctamente TF-IDF, PyTorch, Batch Normalization, Dropout, Adam, CrossEntropyLoss y Early Stopping. El trabajo demuestra cómo integrar técnicas clásicas de NLP con modelos de Deep Learning para resolver un problema real de clasificación textual.